K-Nearest Neighbors

a supervised algorithm used for both classification and Regression.

an instance based algorithm means it doesn't explicitly learn anything during training.

It memorize the train data and for a new data make decisions based comparing with other values.

How KNN works

① choose K - number neighbors need to consider.
② calculate the distance from the new data point to every point in the training data.
③ sort all the points in the training set based on their distance from new data point.
④ Take top K closest points.
⑤ Vote (classification) or average (Regression)
→ classification: majority class among the K neighbors.
→ Regression: mean/median value of K neighbors.

*Distance metrices:

Euclidean Distance (mostly used)

$$d(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

x and y are 2 points

#Manhattan Distance

$$d(x,y) = \sum_{i=1}^{n} |x_i - y_i|$$

Minkowski Distance

$$d(x,y) = \left(\sum_{i=1}^{n} (x_i - y_i)^p \right)^{1/p}$$

if $p=2 \rightarrow$ Euclidean
if $p=1 \rightarrow$ Manhattan

For text/high dimension data we use cosine similarity.

$$Sim(x,y) = \frac{x \cdot y}{\|x\| \|y\|}$$

⇒ How to select value of K.

Heuristic Approach

$K = \sqrt{\text{Total samples}}$

For classification avoid even value of K.

Experimentation

① use grid search cv with cross val.

② use loop with different k values and take the best one based on test accuracy.

KNN Regression:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^{k} y_i$$

It weighted

$$\hat{y} = \frac{\frac{1}{k} \sum_{i=1}^{k} \frac{1}{d_i} y_i}{\frac{1}{k} \sum_{i=1}^{k} \frac{1}{d_i}}$$

$$\hat{y} = \frac{\sum_{i=1}^{k} \frac{1}{d_i} y_i}{\sum_{i=1}^{k} \frac{1}{d_i}}$$

⇒ overfitting and underfitting in KNN

underfit

K is too large

It takes in account to many points even those are far away.

The decision boundary becomes too smoth. that it cannot seperate classes.

Overfit: K is too small [k=1]

It classify each points based on the nearest neighbor only.

Even outliers and noise will influence prediction.

How to handle overfit and underfit

① use cross-validation

② use distance weighting. Closer neighbor will have more weight.

③ Always standardize or normalize data before using KNN.

A non-parametric model is one that does not assume a fixed functional form for the mapping function from input and output.

→ KNN does not assume any underlying distribution like normal.
→ It doesn't learn any parameters.
→ KNN memorizes the training data instead of learning.

A lazy learner delays the learning process until a query is made. It store the train data and doesn't generalize while training.

Limitations (where KNN fails)

① Large dataset
② High dimensional data (curse of dimentionality)
③ If a lot outliers
④ Non Homgenious Feature scale. (data in different scale)
⑤ Imbalance data.
⑥ Not work interance.